

16 > Write, simulate, verify

- Signals propagate
- Flip-flops sample on clock edges

This is why testbenches exist. You don't "print debug logs" in RTL; you drive signals, advance time, and inspect waveforms.

Also, most of industry has moved toward SystemVerilog as the unified standard for design + verification, standardized as IEEE 1800. But you can start with Verilog-2001 style and still learn 90% of the mindset.

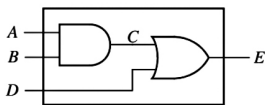
Your starter toolbox: simulator + waveform viewer + editor

You need three things:

1. A simulator (runs RTL and testbenches)
2. A waveform viewer (so you can see signal behavior over time)
3. An editor (VS Code is fine; any text editor works)

Recommended beginner stack:

- iverilog + vvp + GTKWave
Optional "level up":
- Verilator when you want speed and more compiler-like strictness



```
module two_gates (A, B, D, E);  
output E;  
input A, B, D;  
wire C;  
    assign C = A && B; // concurrent  
    assign E = C || D; // statements  
endmodule
```

Setup Lab: install a simulator and waveform viewer

Option A (fastest): use packaged installs

Icarus Verilog's official docs recommend using prepackaged versions when available.

Verilator also notes package managers are easiest (though not always newest).

Option B (Windows): use the Icarus Verilog installer (beginner-friendly)

On Windows, the most painless route is the prebuilt installer packages. One commonly referenced installer source is the MinGW-compiled Icarus Verilog packages, which also include GTKWave in recent releases.

(Also: the MIT FPGA course notes explicitly recommend using an installer on Windows and enabling "add to PATH," and mention GTKWave can be installed alongside.)

Step-by-step: verify your install (all OS)

Do this once and you'll never doubt your setup again.

1. Open a terminal / PowerShell
2. Run:

```
iverilog -V  
vvp -V  
gtkwave --version
```
3. If these print versions instead of "command not found," you're ready.

GTKWave is explicitly designed for post-mortem debugging by analyzing dumpfiles like standard Verilog VCD, so it's perfect for our workflow.

Project structure that keeps you sane

Create a folder like this:

```
ft_semiconductors101/  
  ch7_verilog/  
    src/  
      adder4.v  
      counter.v  
      pwm.v  
      uart_tx.v  
    tb/  
      tb_adder4.v  
      tb_counter.v  
      tb_pwm.v  
      tb_uart_tx.v  
    waves/  
    build/
```

Why this matters:

- You'll have multiple modules + testbenches quickly
- You'll generate wave files (VCD) often
- You'll want clean separation between "design" and "verification"

Your first RTL: a 4-bit adder (and why it matters)

RTL module: adder4.v

```
module adder4 (  
    input  [3:0] a,  
    input  [3:0] b,  
    input      cin,  
    output [3:0] sum,  
    output      cout  
);  
    assign {cout, sum} = a + b + cin;  
endmodule
```